

STREAMS — How << and >> Work

The >> operator reads the **next valid token** from input:

- Token validity based on variable type being read
- Tokens always separated by whitespace
- Whitespace is **never** read (spaces, newlines)
- Even std::string ignores whitespace with >>

Operator Signature:

```
std::ostream& std::ostream::operator<<(const T& data) {
    // output data
    return *this; // returns reference to stream
}
```

STREAMS — Chaining << and >>

```
std::cout << a << " + " << b << " = " << sum << "\n";
```

- **Leftmost** operation evaluated first
- Each << prints right-hand operand, then **returns reference** to left-hand stream (std::cout)
- Returned reference used for next << in chain

- Equivalent to: std::cout.operator<<(a)
- If stream fully consumed or invalid input → stream enters **fail state**

STREAMS — Overloading for Custom Classes

Why overload?

- Custom classes don't work with cin/cout by default
- Enables chaining: cout << obj1 << obj2

- Control over how objects are formatted
- More readable, consistent code

Two approaches:

- **Member function** — called directly on the object (tied to class)
- **Non-member function** — used for cin/cout (RECOMMENDED for << and >>)
- Reason: you don't control std::cout (it's part of STL)
- Allows left-hand operand to be a stream object

STREAMS — Member Function Overloading

- Operator called directly on the object itself
- Member functions auto-bound to left-hand operand
- Useful when operation is tied to the class logic

FRIEND FUNCTIONS & CLASSES

A **friend function** is a non-member function that can access private/protected members:

- Can be standalone functions, entire classes, or member functions of other classes

Declaring a friend function:

```
class MyClass {
    friend void setX(MyClass&, int); // friend declaration
};
```

Declaring a friend class:

```
class ClassOne {
    friend class ClassTwo; // all ClassTwo methods are friends
};
```

Rules of friendship:

- **Must be explicit** — A must declare B as friend, not assumed
- **Not symmetric** — A friend of B ≠ B friend of A
- **Not transitive** — A friend of B, B friend of C ≠ A friend of C

Non-member friend function example:

```
class Rectangle {
    int width, height;
    friend bool compareArea(const Rectangle&,
                           const Rectangle&);

public:
    Rectangle(int w, int h) : width(w), height(h){}
};

bool compareArea(const Rectangle& r1,
                 const Rectangle& r2) {
    return r1.width*r1.height > r2.width*r2.height;
}
```

■ Use when operation is logically external to class

■ Better symmetry: compareArea(r1,r2) vs r1.compareArea(r2)

MEMBER vs NON-MEMBER OVERLOADING

Feature	Member	Non-Member
Definition	Inside class	Outside class
Tied to object	Yes	No
this pointer	Has access	No access
Access scope	All members	Public only (*friend)
Arguments	One fewer (implicit this)	All explicit
Call syntax	obj.func()	func(lhs, rhs)
Best for	Instance ops	Symmetric ops / streams

CLASSES — Key Concepts

- **Private members** — only accessible within the class
- **Constructor** — initialises the object when created
- **Member initialiser list** — preferred way to initialise members
- MyClass(int x, int y) : x(x), y(y) {}
- **Destructor** (~MyClass) — called when object destroyed, used for cleanup
- **Static members** — shared across ALL instances of the class
- static int objectCount; // shared by all objects
- Item::objectCount++; // access via class name
- **Virtual functions** — enable polymorphism (correct override called via base pointer)
- virtual std::string formatInfo(); // in base class
- std::string formatInfo() override; // in derived class
- **Inheritance** — derived class calls base constructor in initialiser list
- SoccerTeam(string name, int won) : Team(name), won(won) {}

```

class WaitingLine {
public:
    std::queue line;
    WaitingLine& operator<<(const std::string& name) {
        this->line.push(name);
        return *this; // enables chaining
    }
    WaitingLine& operator>>(std::string& front) {
        front = line.front();
        this->line.pop();
        return *this;
    }
};

WaitingLine wl;
wl << "Alice" << "Bob"; // unconventional <<
wl >> name;              // unconventional >>

```

STREAMS — Non-Member Function Overloading

Signature pattern:

```

std::ostream& operator<<(std::ostream& out,
                        const MyClass& obj) {
    out << obj.member; // access via friend
    return out;        // MUST return stream
}

std::istream& operator>>(std::istream& in,
                        MyClass& obj) {
    in >> obj.member;
    return in;
}

```

Full Example (Coordinate class):

```

class Coordinate {
private:
    int x, y;
public:
    Coordinate(int x=0, int y=0) { this->x=x; this->y=y; }
    friend std::ostream& operator<<(std::ostream& out,
                                    const Coordinate& c);
    friend std::istream& operator>>(std::istream& in,
                                    Coordinate& c);
};

std::ostream& operator<<(std::ostream& out,
                        const Coordinate& c) {
    out << c.x << ", " << c.y << std::endl;
    return out;
}

std::istream& operator>>(std::istream& in,
                        Coordinate& c) {
    in >> c.x >> c.y;
    return in;
}

Coordinate c1;
std::cin >> c1;
std::cout << "Location: " << c1;

```

POINTERS

- **Pointer** — a variable that stores a memory address

```
int* ptr;           // declares a pointer to int
int x = 5;
ptr = &x;           // ptr holds address of x
int val = *ptr;     // dereference: val = 5
```

- **Arrow operator** — dereference pointer AND access member
obj->method(); // same as (*obj).method()

Dynamic Memory Allocation:

```
int* p = new int(8); // allocate on heap
delete p;            // free memory
p = nullptr;         // safe guard
```

- **new** — allocates memory on the heap, returns pointer

- **delete** — frees the heap memory pointed to
- **delete nullptr** — always safe, does nothing

Common Pointer Errors:

- **Dangling pointer** — using pointer after delete

```
delete p;
int v = *p; // ERROR: undefined behaviour
// Fix: set p = nullptr after delete
```

- **Invalid deref** — dereferencing pointer not from new

```
int x = 5; int* p = &x;
delete p; // ERROR: not heap allocated
```

- **Double-delete** — deleting same pointer twice

```
delete p; delete p; // ERROR: undefined behaviour
```

Correct Pattern:

```
MyClass* obj = new MyClass(args);
obj->method(); // use
delete obj;    // free exactly once
obj = nullptr; // nullify immediately
```

MEMORY MANAGEMENT

- **Destructor must delete all heap-allocated members**

```
Basket::~Basket() {
    for (Item* item : items) {
        delete item; // free each Item
    }
    items.clear(); // clear vector
}
```

- **Static counter tracks live objects (decremented by destructor)**

```
// Item.h
static int objectCount;
// Item.cpp — constructor
Item::objectCount++;
// Item.cpp — destructor
Item::~objectCount--;
```

- **After delete basket: all item objects freed $\Rightarrow$ objectCount = 0**

■ **Memory leaks occur when new is called without matching delete**

VECTORS

- **1D vector** — dynamic array of elements

```
std::vector v;
v.push_back(5); // add element
v.size();       // number of elements
v[0];           // access by index
for (int x : v) { } // range-based loop
```

- **2D vector** — vector of vectors

```
std::vector< grid>
for (auto& row : grid)
    for (auto& word : row)
        process(word);
```

- **vector of pointers** — stores addresses of heap objects

CONSTRUCTOR ARGUMENTS vs PARAMETERS

- **Parameters** — placeholders defined in the constructor declaration

```
// name, points, gamesWon, gamesLost are PARAMETERS
AFLTeam(std::string name, int points,
        int gamesWon, int gamesLost);
```

- **Arguments** — actual values supplied when creating the object

```
// "West Coast Eagles", 90, 0, 0 are ARGUMENTS
AFLTeam team("West Coast Eagles", 90, 0, 0);
```

■ **Parameters** are the placeholders — live in the declaration

■ **Arguments** are the real values — supplied at object creation

LOOPS

- **For loop** — fixed number of iterations

```
for (int i = 0; i < n; i++) { }
```

- **Range-based for** — iterate all elements

```
for (auto& item : vector) { }
```

- **While loop** — condition checked each iteration

```
while (stream >> token) { } // reads until fail
```

- **Nested loops** — used for 2D vectors

```
for (auto& row : matrix)
    for (auto& word : row) { }
```

- **Main condition loop** — prime checking pattern

```
for (int i=2; i*i <= num; i++)
    if (num % i == 0) return false;
```

STRINGS & STRINGSTREAM

- **std::string** — text data type

```
std::string s = "hello";
s.length();    // number of chars
s + " world";  // concatenation
```

- **stringstream** — base text like a stream

```
std::stringstream ss("electronics Laptop");
std::string label, item;
ss >> label >> item; // reads token by token
```

- **getline** — reads entire line including spaces

```
std::string line;
while (std::getline(ss, line)) {
    // process each line
}
```

- **Output formatting** — control field widths

```
#include
std::cout << std::left
          << std::setw(19) << name
          << std::setw(9) << points;
```

■ **setw(n)** — sets minimum field width to n characters

■ **left** — left-justify content within the field

■ **fixed / setprecision(n)** — control decimal places

INHERITANCE & POLYMORPHISM

- **Base class** — parent, defines common interface

- **Derived class** — child, inherits and can extend/override

```
std::vector items;
items.push_back(new Item("milk", 3.99, 1));
for (Item* p : items) delete p; // cleanup
```

- **erase()** — remove element at iterator position

```
auto it = items.begin();
while (it != items.end()) {
    if ((*it)->name == target) {
        delete *it;
        it = items.erase(it); // erase shifts rest
    } else { ++it; }
}
```

```
class SoccerTeam : public Team {
private:
    int won, lost, drawn;
public:
    SoccerTeam(string name, string coach,
                int played, int won,
                int lost, int drawn)
        : Team(name, coach, played), // base constructor
          won(won), lost(lost), drawn(drawn) {}
    std::string formatInfo() override; // override
};
```

Virtual — marks function for override; if base not overridden

```
// In Team.h
virtual std::string formatInfo();
// In SoccerTeam.h
std::string formatInfo() override;
```

- **Polymorphism** — base pointer calls correct derived function

```
Team* t = new SoccerTeam(...);
t->formatInfo(); // calls SoccerTeam version
```

■ Without virtual → always calls base class version

■ With virtual → calls correct derived version at runtime

QUICK REFERENCE — COMMON PATTERNS

- Check if item exists in vector:

```
bool found = false;
for (auto& x : vec)
    if (x == target) { found = true; break; }
```

- Convert an array into a vector from stringstream

```
std::string line, label, item;
while (std::getline(ss, line)) {
    std::stringstream ls(line);
    ls >> label >> item;
    if (label == target) result.push_back(item);
}
```

- Format string with padding (ostringstream)

```
std::ostringstream oss;
oss << std::left << std::setw(19) << name
    << "| " << std::setw(9) << points;
return oss.str();
```